

Initial data analysis: Google Year in Search (trends.csv)

Objective. Load the dataset, inspect it with the standard pandas methods and write down what each output actually says. Nothing gets modelled here. The job is to find out what we have, what is broken in it and what has to be cleaned before any analysis is worth running.

Method. One function per section. Show the output, read the output. Problems found along the way pile up into a cleaning checklist in section 11.

1. Imports and loading

```
In [1]: import pandas as pd
import numpy as np

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 140)

print('pandas version:', pd.__version__)
```

pandas version: 2.2.3

```
In [2]: df = pd.read_csv('trends.csv')
print('File loaded successfully.')
```

File loaded successfully.

No parser errors and no dtype warnings. That tells us two things. The column count is the same on every line, and the quoting is correct.

The quoting matters more than it sounds. Some queries have commas inside them, and at least one has embedded quote characters: Toys "R" Us . If the quoting were broken, pandas would split that row on the internal comma, find six fields where it expected five and throw a tokenising error. It did not, so the file is sound at the character level before we look at a single value.

2. shape: how much data is there?

```
In [3]: print('Shape (rows, columns):', df.shape)
print('Number of rows   :', df.shape[0])
print('Number of columns:', df.shape[1])
print('Total cells      :', df.size)
```

```
Shape (rows, columns): (26955, 5)
Number of rows      : 26955
Number of columns: 5
Total cells         : 134775
```

26,955 rows, 5 columns. Long and narrow, which is what you get when each row is one observation.

The row count is worth checking against the structure. $26,955 / 5 = 5,391$ exactly. Every ranking here is a top-5 list, so a clean file has to hold a multiple of 5 rows. It does.

That is a weak check though. A file could lose one row and gain another and still divide by 5. Section 10 tests the structure properly.

3. `head()` and `tail()`: what does a record look like?

```
In [4]: df.head(10)
```

```
Out[4]:
```

	location	year	category	rank	query
0	Global	2001	Consumer Brands	1	Nokia
1	Global	2001	Consumer Brands	2	Sony
2	Global	2001	Consumer Brands	3	BMW
3	Global	2001	Consumer Brands	4	Palm
4	Global	2001	Consumer Brands	5	Adobe
5	Global	2001	Men	1	Nostradamus
6	Global	2001	Men	2	Osama bin Laden
7	Global	2001	Men	3	Eminem
8	Global	2001	Men	4	Michael Jackson
9	Global	2001	Men	5	Howard Stern

Each row is one entry in a ranked list. Location, year, category, rank 1 to 5, and the query that placed there.

The first ten rows are all `Global` and `2001`, running through `Consumer Brands` and then `Men`, rank going 1 to 5 inside each. So the sort order is location, then year, then category, then rank. Rows come in blocks of five and one block is one complete top-5 list. Hold onto that. It is the rule the whole file is built on and section 10 leans on it.

```
In [5]: df.tail(10)
```

Out[5]:

	location	year	category	rank	query
26945	Vietnam	2020	Như Thế Nào?	1	Cúng giao thừa như thế nào
26946	Vietnam	2020	Như Thế Nào?	2	Vụ án Hồ Duy Hải như thế nào
26947	Vietnam	2020	Như Thế Nào?	3	Bầu cử tổng thống mỹ như thế nào
26948	Vietnam	2020	Như Thế Nào?	4	Tuấn khí bị bắt như thế nào
26949	Vietnam	2020	Như Thế Nào?	5	Gấu đi như thế nào
26950	Vietnam	2020	Là Gì?	1	Virus Corona là gì
26951	Vietnam	2020	Là Gì?	2	Miễn thị thực là gì
26952	Vietnam	2020	Là Gì?	3	Đầu cắt moi là gì
26953	Vietnam	2020	Là Gì?	4	Bệnh bạch hầu là gì
26954	Vietnam	2020	Là Gì?	5	Đông Lào là gì

The tail is Vietnam, 2020, with categories in Vietnamese. Là Gì? is "what is".
Như Thế Nào? is "how".

So basically the category labels are not standardised. Each country wrote them in its own language, which means one concept gets stored under many different strings, which means grouping by category will shatter into fragments. Section 9 measures exactly how badly.

The queries are non-English too and the Vietnamese diacritics survived intact, so the file is UTF-8 and pandas decoded it correctly. Any text processing later has to be Unicode-safe, not ASCII.

4. columns and dtypes : what are the fields?

```
In [6]: print('Column names:')
print(list(df.columns))
```

```
Column names:
['location', 'year', 'category', 'rank', 'query']
```

```
In [7]: df.dtypes
```

```
Out[7]: location    object
year              int64
category          object
rank              int64
query             object
dtype: object
```

Five columns. Names are already lowercase with no spaces or trailing whitespace. Nothing to rename.

pandas reads `location`, `category` and `query` as `object`, meaning Python strings. `year` and `rank` come in as `int64`.

Both integers are correctly typed and both are traps. `year` is a time label. `rank` is an ordinal position from 1 to 5. Neither is a measurement, so arithmetic on them produces a number that means nothing, and pandas will compute it without complaining. Section 6 shows what that looks like when it happens.

A cleaned version should cast `location` and `category` to `category` dtype. Both hold very few distinct values against 26,955 rows, so the memory drop is large.

5. `info()`: types, non-null counts and memory

```
In [8]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26955 entries, 0 to 26954
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   location    26955 non-null  object
1   year        26955 non-null  int64
2   category    26955 non-null  object
3   rank        26955 non-null  int64
4   query       26955 non-null  object
dtypes: int64(2), object(3)
memory usage: 1.0+ MB
```

`info()` repeats the shape and types and adds two facts.

Non-null count is 26,955 on all five columns, matching the row count exactly. No missing values anywhere. For real data that is strange, and the reason is provenance. This is a published, curated ranking, not raw collected data. Google built the top-5 lists and then released them, so there was never an opportunity for a value to go missing. Section 7 checks that claim rather than taking this count at face value.

Memory sits around 1 MB. Nothing here needs chunking or optimising.

6. `describe()`: summary statistics

```
In [9]: df.describe()
```

Out[9]:

	year	rank
count	26955.000000	26955.000000
mean	2015.243369	3.000000
std	3.564683	1.41424
min	2001.000000	1.00000
25%	2013.000000	2.00000
50%	2016.000000	3.00000
75%	2018.000000	4.00000
max	2020.000000	5.00000

`describe()` defaults to numeric columns, so it gives us `year` and `rank`. Read both carefully.

`rank`: mean exactly 3.000000, min 1, max 5, standard deviation 1.41424. None of that tells us anything about search behaviour. Every block holds one of each rank 1 through 5, so the distribution is uniform by construction. A uniform 1 to 5 always has a mean of 3. Its standard deviation is $\sqrt{2} = 1.41421$, and ours matches to four decimal places.

Put another way, running `describe()` on `rank` is like taking the average of the numbers on a dartboard. You get 10.5 every time, and it says nothing about anyone's aim. What these numbers do confirm is that the file is structurally intact.

`year`: 2001 to 2020, a 20-year span. Mean 2015.24 and median 2016 both sit well above the midpoint of that range, so rows are not spread evenly. Recent years carry far more of them, because Google kept adding countries and categories over time. That skew propagates. More countries covered means more rows in that year, which means any raw count comparison across years is measuring coverage rather than behaviour.

Two numeric columns, two sets of statistics that are either tautological or misleading. Worth noticing before anyone quotes them as findings.

```
In [10]: df.describe(include='object')
```

Out[10]:

	location	category	query
count	26955	26955	26955
unique	83	2450	18431
top	United States	People	Paul Walker
freq	2070	760	84

`include='object'` gives the text columns, and this is where the actual information lives.

`location` has 83 unique values, most frequent `United States` at 2,070 rows. `category` has 2,450 unique values, top `People` at 760. `query` has 18,431 unique values, top `Paul Walker` at 84.

2,450 categories against 83 locations and 20 years is the number that does not belong. Section 9 goes after it.

`Paul Walker` at 84 is not an error. He died in November 2013 and got searched worldwide, so he lands in many countries in the same year. Queries repeating across locations is normal for this file.

Explicit mean, median, min, max and standard deviation

```
In [11]: num = df[['year', 'rank']]

stats = pd.DataFrame({
    'mean' : num.mean(),
    'median': num.median(),
    'min' : num.min(),
    'max' : num.max(),
    'std' : num.std(),
})
stats
```

Out[11]:

	mean	median	min	max	std
year	2015.243369	2016.0	2001	2020	3.564683
rank	3.000000	3.0	1	5	1.414240

Same figures, in the form the task asks for. Reading does not change. `rank` is uniform by design and `year` leans recent. Median `year` of 2016 against a range midpoint of 2010.5 puts a number on the lean.

7. `isnull().sum()`: missing values

```
In [12]: print('Missing values per column:')
print(df.isnull().sum())
print()
print('Total missing values in the dataset:', df.isnull().sum().sum())
```

Missing values per column:

```
location    0
year        0
category    0
rank        0
query       0
dtype: int64
```

Total missing values in the dataset: 0

Zero missing values in every column. Nothing to impute, nothing to drop.

A zero can also mean the nulls are wearing a disguise. The next cell checks the usual ones: empty strings, whitespace-only strings and text placeholders like `NA`, `null`, `-` and `unknown`.

```
In [13]: placeholders = ['', 'na', 'n/a', 'nan', 'null', 'none', '-', '--', 'unknown']

for col in ['location', 'category', 'query']:
    s = df[col].astype(str)
    blanks = (s.str.strip() == '').sum()
    fake    = s.str.strip().str.lower().isin(placeholders).sum()
    print(f'{col:9s} | empty or whitespace-only: {blanks:4d} | placeholder t
```

```
location | empty or whitespace-only:    0 | placeholder text:    0
category | empty or whitespace-only:    0 | placeholder text:    0
query    | empty or whitespace-only:    0 | placeholder text:    0
```

Every count zero. The missing values are genuinely absent, not hidden. On this one measure the file is in better shape than most real data.

8. `df.duplicated().sum()`: duplicate records

```
In [14]: print('Number of fully duplicated rows:', df.duplicated().sum())
print('Percentage of the dataset      :', round(100 * df.duplicated().sum()
```

```
Number of fully duplicated rows: 10
Percentage of the dataset      : 0.0371 %
```

Ten duplicated rows. 0.037 percent of the file. Small enough to walk past, big enough to distort any count or frequency computed later.

The number on its own will not say whether these are ten scattered accidents or one systematic fault. Next cell prints every duplicated row with `keep=False` so the first occurrence shows up too.

```
In [15]: dupes = df[df.duplicated(keep=False)].sort_values(['location', 'year', 'cate
dupes
```

Out[15]:

	location	year	category	rank	query
19995	Kazakhstan	2018	Жылдың фильмі/ Фильм года	1	Веном
20000	Kazakhstan	2018	Жылдың фильмі/ Фильм года	1	Веном
19996	Kazakhstan	2018	Жылдың фильмі/ Фильм года	2	Мстители: Война бесконечности
20001	Kazakhstan	2018	Жылдың фильмі/ Фильм года	2	Мстители: Война бесконечности
19997	Kazakhstan	2018	Жылдың фильмі/ Фильм года	3	Бизнес по-казахски в Америке
20002	Kazakhstan	2018	Жылдың фильмі/ Фильм года	3	Бизнес по-казахски в Америке
19998	Kazakhstan	2018	Жылдың фильмі/ Фильм года	4	Монстры на каникулах 3
20003	Kazakhstan	2018	Жылдың фильмі/ Фильм года	4	Монстры на каникулах 3
19999	Kazakhstan	2018	Жылдың фильмі/ Фильм года	5	Дэдпул 2
20004	Kazakhstan	2018	Жылдың фильмі/ Фильм года	5	Дэдпул 2
25395	Kenya	2020	Trending How To (Tech)	1	How to fill Tpad online
25400	Kenya	2020	Trending How To (Tech)	1	How to fill Tpad online
25396	Kenya	2020	Trending How To (Tech)	2	How to create a website
25401	Kenya	2020	Trending How To (Tech)	2	How to create a website
25397	Kenya	2020	Trending How To (Tech)	3	How to connect a phone to TV
25402	Kenya	2020	Trending How To (Tech)	3	How to connect a phone to TV
25398	Kenya	2020	Trending How To (Tech)	4	How to make my electric plate cooker work
25403	Kenya	2020	Trending How To (Tech)	4	How to make my electric plate cooker work
25399	Kenya	2020	Trending How To (Tech)	5	How to take a screenshot on a laptop
25404	Kenya	2020	Trending How To (Tech)	5	How to take a screenshot on a laptop

Not scattered at all. They are two complete blocks, each repeated once:

1. Kazakhstan, 2018, Жылдың фильмі/Фильм года (Kazakh and Russian for "film of the year"), ranks 1 to 5.
2. Kenya, 2020, Trending How To (Tech) , ranks 1 to 5.

Each appears twice in full. Twenty rows where there should be ten.

The shape of the fault points straight at the cause. Ten random duplicate rows would mean sloppy data entry. Two entire top-5 lists repeated cleanly means the file was built by appending per-country blocks and someone appended two of them twice. That is an ingestion error, not a data error, which is good news: `drop_duplicates()` handles it safely because the repeated rows match on all five columns.

Look at the category name too. `Trending How To (Tech)` has a leading space and a double space in the middle. Section 9 comes back to that.

```
In [16]: print('Rows before dropping duplicates:', len(df))
print('Rows after dropping duplicates :', len(df.drop_duplicates()))
print('Rows removed                :', len(df) - len(df.drop_duplicates()))
print()
print('Is the cleaned row count divisible by 5?', len(df.drop_duplicates())
```

Rows before dropping duplicates: 26955

Rows after dropping duplicates : 26945

Rows removed : 10

Is the cleaned row count divisible by 5? True

26,945 rows after the drop, still divisible by 5. The block structure survived the fix, which is a decent sign we took out the right rows.

9. `nunique()` : unique values and cardinality

```
In [17]: print('Unique values per column:')
print(df.nunique())
```

Unique values per column:

location 83

year 20

category 2450

rank 5

query 18431

dtype: int64

Against 26,955 total rows, four of these five counts behave and one does not.

`rank` at 5 matches top-5 lists. `year` at 20 is a continuous run from 2001 to 2020, confirmed next cell. `location` at 83 is countries plus a `Global` aggregate. `query` at 18,431 is high but sensible, because most searches belong

to one country in one year and the repeats are global events and celebrity deaths.

`category` at 2,450 does not fit. 83 locations, 20 years and a few dozen sensible category types would be the expectation. 2,450 is the anomaly of this dataset.

```
In [18]: print('Year range:', df.year.min(), 'to', df.year.max())
print('Distinct years:', df.year.nunique())
missing_years = set(range(df.year.min(), df.year.max() + 1)) - set(df.year.u
print('Gaps in the year sequence:', missing_years if missing_years else 'nor
print()
print('Rows per year:')
print(df.year.value_counts().sort_index())
```

Year range: 2001 to 2020

Distinct years: 20

Gaps in the year sequence: none

Rows per year:

year	
2001	60
2002	110
2003	155
2004	145
2005	10
2006	75
2007	60
2008	895
2009	445
2010	90
2011	890
2012	2225
2013	2890
2014	2320
2015	2790
2016	3040
2017	2610
2018	2560
2019	2605
2020	2980

Name: count, dtype: int64

2001 to 2020, no gaps. The row counts confirm what the mean of `year` already hinted at. Early 2000s give a few hundred rows each and the 2010s give thousands. Coverage grew, it did not hold steady, so a year-on-year comparison is comparing samples of very different sizes.

```
In [19]: print('Rank distribution:')
print(df['rank'].value_counts().sort_index())
```

Rank distribution:

rank

1	5391
2	5391
3	5391
4	5391
5	5391

Name: count, dtype: int64

Each rank appears exactly 5,391 times. That balance only happens if almost every block is a complete 1-to-5 list. Section 10 checks it directly instead of inferring it from here.

```
In [20]: print('Number of locations:', df.location.nunique())
print()
print('Top 10 locations by row count:')
print(df.location.value_counts().head(10))
print()
print('Bottom 10 locations by row count:')
print(df.location.value_counts().tail(10))
```

Number of locations: 83

Top 10 locations by row count:

location

United States	2070
Global	1135
Japan	765
Canada	690
Brazil	675
France	630
United Kingdom	590
Finland	555
Mexico	550
Thailand	525

Name: count, dtype: int64

Bottom 10 locations by row count:

location

Zimbabwe	30
Ecuador	20
Myanmar (Burma)	15
Venezuela	15
Sri Lanka	10
Honduras	5
El Salvador	5
Dominican Republic	5
Kuwait	5
Sudan	5

Name: count, dtype: int64

United States leads at 2,070 rows, then Global at 1,135. The tail is countries with a handful of rows each.

`Global` needs flagging. It is an aggregate, not a country, and it sits in the same column as the real countries. So a `groupby('location')` counts every global-level row twice, once inside `Global` and once inside whichever country also reported it. Nothing in the data marks it as different, which is exactly why this kind of mixed-granularity bug survives into published results.

The category problem

```
In [21]: print('Total distinct categories:', df.category.nunique())
print()
print('Top 20 categories by row count:')
print(df.category.value_counts().head(20))
```

Total distinct categories: 2450

Top 20 categories by row count:

category	
People	760
Searches	620
Movies	330
TV Shows	305
Películas	250
Songs	215
Recipes	175
What is...?	175
How to...	170
News	150
Events	125
Athletes	125
Deportistas	115
Acontecimientos	115
Cómo	110
Sports	100
Cómo...	100
Búsquedas	100
Fastest Rising Searches	100
Canciones	100

Name: count, dtype: int64

The top of the list gives it away. `Movies` has 330 rows. `Películas`, which is the same word in Spanish, has 250 and sits in a separate group. `Athletes` has 125, `Deportistas` has 115. `How to...` and `Cómo` are one idea in two languages.

Each country recorded the labels in its own language, so a single concept splits across many strings, which means the true number of categories is far smaller than 2,450 and every one of those splits is invisible to `groupby`. Next cell measures the damage.

```
In [22]: cat_locs = df.groupby('category')['location'].nunique()
cat_rows = df.category.value_counts()
```

```
print('Categories used in only ONE location :', (cat_locs == 1).sum(), 'of',
print('Categories with 5 rows or fewer      :', (cat_rows <= 5).sum(), 'of',
print()
print('Share of categories that are single-location:',
      round(100 * (cat_locs == 1).sum() / df.category.nunique(), 1), '%')
```

Categories used in only ONE location : 2132 of 2450
 Categories with 5 rows or fewer : 1656 of 2450

Share of categories that are single-location: 87.0 %

2,132 of the 2,450 categories appear in only one location. 1,656 appear in a single block of five rows. Roughly 87 percent of the labels belong to exactly one country.

So this is not one category system with 2,450 entries. It is 83 national labelling schemes stacked into one column. Grouping on it as it stands returns over two thousand groups, most of them five rows deep, and every cross-country comparison built on that is meaningless. The column has to be mapped to a standard English taxonomy first. That mapping is the biggest single piece of cleaning this dataset needs.

```
In [23]: for col in ['location', 'category', 'query']:
          s = df[col].astype(str)
          ws = (s != s.str.strip()).sum()
          dblsp = s.str.contains(' ', regex=False).sum()
          casefold_gap = s.nunique() - s.str.lower().nunique()
          print(f'{col:9s} | leading/trailing whitespace: {ws:4d} | internal double
```

```
location | leading/trailing whitespace:    0 | internal double spaces:    0
| values differing only by case:    0
category | leading/trailing whitespace: 175 | internal double spaces:   30
| values differing only by case:   76
query    | leading/trailing whitespace:    0 | internal double spaces:    4
| values differing only by case:  446
```

`category` holds 175 values with leading or trailing whitespace and 30 with internal double spaces, `Trending How To (Tech)` from the duplicate block among them. 76 category values and 446 query values differ from some other value only by capitalisation.

`.str.strip()` and a whitespace collapse fix these cheaply. Run them first. Stray whitespace inflates every unique count, so cleaning it before the taxonomy work means the taxonomy work starts from an honest number.

10. Structural integrity check

Everything so far points at one rule. Every combination of location, year and category should hold exactly five rows, ranked 1 to 5. This section tests the rule instead of assuming it.

```
In [24]: group_sizes = df.groupby(['location', 'year', 'category']).size()

print('Total groups (location, year, category):', len(group_sizes))
print()
print('Distribution of group sizes:')
print(group_sizes.value_counts())
print()
bad = group_sizes[group_sizes != 5]
print('Groups that do NOT have exactly 5 rows:', len(bad))
print(bad)
```

Total groups (location, year, category): 5389

Distribution of group sizes:

5 5387

10 2

Name: count, dtype: int64

Groups that do NOT have exactly 5 rows: 2

location	year	category	
Kazakhstan	2018	Жылдың фильмі/Фильм года	10
Kenya	2020	Trending How To (Tech)	10

dtype: int64

5,387 groups hold exactly five rows. Two hold ten. Those two are the Kazakhstan and Kenya blocks from section 8.

That is the useful part. This check was built on a completely different idea from `deduplicated()`, it came at the file from the other end, and it found the same two faults and nothing else. Two independent methods agreeing is much stronger evidence than either one alone. The rule holds everywhere except the known duplicates.

```
In [25]: dup_rank = df.groupby(['location', 'year', 'category'])['rank'].apply(lambda x: x.duplicated())
print('Groups containing a repeated rank:', (dup_rank > 0).sum())

clean = df.drop_duplicates()
sizes_clean = clean.groupby(['location', 'year', 'category']).size()
print('After dropping duplicates, groups not equal to 5:', (sizes_clean != 5).sum())
```

Groups containing a repeated rank: 2

After dropping duplicates, groups not equal to 5: 0

No group holds a repeated rank once the duplicates are gone, and every group is back to five rows. `drop_duplicates()` repairs the structure completely.

```
In [26]: years_per_loc = df.groupby('location')['year'].nunique()

print('Years covered per location:')
```

```
print(years_per_loc.describe())
print()
print('Locations present in all 20 years:', (years_per_loc == 20).sum())
print('Locations present in only 1 year :', (years_per_loc == 1).sum())
```

Years covered per location:

count	83.000000
mean	9.060241
std	4.049539
min	1.000000
25%	6.000000
50%	10.000000
75%	12.000000
max	20.000000

Name: year, dtype: float64

Locations present in all 20 years: 1

Locations present in only 1 year : 8

Coverage is uneven. The median location shows up in 10 of the 20 years. Some appear in all 20. Some appear in one.

That makes this an unbalanced panel. Countries entered and left the published rankings at different times, so a trend computed across the whole file is partly tracking which countries happened to be covered that year. Think of it as measuring a country's average height using whoever turned up that day. The number moves, but not for the reason you want it to.

This is a property of the data, not an error. It still limits what can honestly be concluded.

11. Overall interpretation

What the dataset is

`trends.csv` holds Google Year in Search rankings. 26,955 rows, five columns, 83 locations, the years 2001 to 2020. Each row is one entry in a top-5 list keyed by location, year, category and rank.

Analytically the file is broad and shallow. Coverage across countries and years is good, but each observation carries four descriptive fields and there is no volume, frequency or search-count anywhere in it. The file records which queries ranked. It never records by how much. A query that won its category by a factor of ten looks identical to one that scraped in. That confines the dataset to rank-based and frequency-based work, and any question needing magnitude cannot be answered from this file at all.

What is in good condition

No missing values in any column and none disguised as blanks or placeholders. Consistent column count and correct quoting throughout, including queries holding commas and embedded quote characters. `year` and `rank` carry correct integer types. Non-Latin scripts decode correctly as UTF-8. The five-rows-per-group rule holds for 5,387 of 5,389 groups.

What needs cleaning

1. Category labels are the main problem. 2,450 distinct values, 2,132 of them in a single location, because each country wrote its labels in its own language. `Movies`, `Películas` and `Filme` sit in three separate groups. The column cannot be grouped on until it is mapped to a standard taxonomy, and that is the largest job on this list.
2. Ten duplicate rows. Two complete top-5 blocks, Kazakhstan 2018 and Kenya 2020, each appearing twice. An append error during file assembly. `drop_duplicates()` removes them and restores the five-row rule.
3. Whitespace in category values. 175 with leading or trailing spaces, 30 with internal double spaces. Strip and collapse before grouping.
4. `Global` sits in `location` next to individual countries at a different level of granularity. Grouping by location without handling it double-counts.
5. `year` and `rank` are integers but neither is a measurement. `rank` is ordinal with a mean pinned at 3.0 by construction. `year` is a label. `describe()` returns numbers for both and those numbers are not findings.
6. Coverage is unbalanced across years and locations. Rows per year climb sharply over the period and the median location appears in 10 of 20 years. Any comparison has to account for that rather than treating the panel as complete.

Cleaning sequence

- | | |
|--|-----------------|
| 1. <code>df.drop_duplicates()</code>
duplicate rows | removes the 10 |
| 2. strip and collapse whitespace on text columns
category values | fixes the 175 |
| 3. map category to a standard English taxonomy
to a usable set | collapses 2,450 |
| 4. separate or flag the Global rows
granularity | resolves mixed |
| 5. cast location and category to category dtype | reduces memory |
| 6. treat year and rank as labels, not measures
meaningless statistics | prevents |

What it can answer once cleaned

With the category column mapped, the file supports questions about which topics dominate searches in a given year, how fast a global event spreads across countries, which queries recur in many locations and how the category mix shifts over the 20-year span. Anything about search volume needs a different source.